

Student Learning Management System

API Design Document

Upgrade2Architect Phase 3 — Version 1.0

REST API | JWT Authentication | JSON | AWS API Gateway | Spring Boot 3.x

Attribute	Value
Document title	Student LMS — API Design Document
Version	1.0
Status	Draft
API style	REST + WebSocket
Base URL	https://api.lms.ctsu2a.com/api
Auth mechanism	JWT Bearer token (OAuth 2.0 compatible)
Content type	application/json
API versioning	URL path versioning: /api/v1/...
Date	22-May-2026

Table of Contents

Table of Contents	2
1. API Overview	4
1.1 Introduction	4
1.2 Base URL and versioning	4
1.3 API design principles	4
1.4 Request and response conventions	4
2. Authentication and Security	6
2.1 Authentication mechanism	6
Token structure.....	6
Authorization header format.....	6
JWT payload claims.....	6
2.2 Role-based access control	6
2.3 Security headers	7
2.4 Rate limiting	7
3. API Endpoint Reference	8
3.1 Authentication endpoints	8
POST /api/auth/register — request body	8
POST /api/auth/register — response (201 Created).....	8
POST /api/auth/login — request body.....	8
POST /api/auth/login — response (200 OK)	8
3.2 Course management endpoints.....	9
GET /api/courses — query parameters.....	9
POST /api/courses — request body.....	9
Course response object.....	10
3.3 Enrolment endpoints	11
POST /api/enrollments — request body.....	11
POST /api/enrollments — response (201 Created)	11
3.4 Assessment and submission endpoints.....	11
POST /api/assessments — request body	12
POST /api/submissions — request body.....	12
POST /api/submissions — response (202 Accepted).....	12
PUT /api/submissions/{id}/grade — request body	12
3.5 Progress endpoints	14
GET /api/progress — response.....	14
PUT /api/progress/lesson-complete — request body	14

- 3.6 Log endpoints..... 14
 - GET /api/logs — query parameters..... 15
- 4. Error Handling..... 16
 - 4.1 Error response format (RFC 7807) 16
 - 4.2 HTTP status codes..... 16
 - 4.3 Validation error response 17
- 5. File Upload (Assignment Submissions)..... 18
 - 5.1 Upload approach 18
 - 5.2 Upload flow 18
 - 5.3 Accepted file types and size limits 18
- 6. WebSocket API (Real-time Notifications)..... 19
 - 6.1 Overview 19
 - 6.2 Connection 19
 - 6.3 Notification message format 19
 - 6.4 Notification types 19
- 7. Pagination and Filtering 21
 - 7.1 Pagination 21
 - Standard list response envelope..... 21
 - Pagination query parameters..... 21
 - 7.2 Filtering conventions 21
- 8. Complete API Quick Reference 22
 - Authentication 22
 - Users 22
 - Courses..... 22
 - Modules and lessons..... 22
 - Enrolment..... 23
 - Assessments..... 23
 - Submissions..... 23
 - Progress..... 23
 - Logs..... 23
- 9. API Changelog..... 25

1. API Overview

1.1 Introduction

The LMS REST API provides a programmatic interface to all platform capabilities. It is built on Spring Boot 3.x, deployed on Amazon ECS Fargate, and exposed via Amazon API Gateway. All communication is over HTTPS (TLS 1.2+). The API follows REST conventions and returns JSON responses conforming to RFC 7807 for errors.

1.2 Base URL and versioning

Environment	Base URL
Production	https://api.lms.ctsu2a.com/api/v1
Staging	https://api-staging.lms.ctsu2a.com/api/v1
Local development	http://localhost:8080/api/v1

API versioning uses URL path prefixing (/api/v1/...). Non-breaking changes (new optional fields) are added without a version bump. Breaking changes increment the version. The previous version is supported for 6 months after a new version is released.

1.3 API design principles

- Resource-oriented URIs: nouns not verbs (e.g. /courses not /getCourses)
- HTTP method semantics: GET=read, POST=create, PUT=update, DELETE=remove
- Consistent JSON casing: camelCase for all field names
- ISO 8601 for all dates and timestamps: 2025-06-15T10:30:00Z
- Pagination on all list endpoints: page and size query parameters
- Idempotency: PUT is idempotent; POST is not
- HATEOAS: not implemented in v1; planned for v2

1.4 Request and response conventions

Convention	Detail
Content-Type header	application/json for all request and response bodies
Accept header	application/json (default; no content negotiation in v1)
Character encoding	UTF-8 for all payloads
Date format	ISO 8601: 2025-06-15T10:30:00Z
Null fields	Omitted from responses (not included as null)
Empty lists	Returned as [] not null

Convention	Detail
Pagination default	page=0, size=20, max size=100
Sort parameter	?sort=createdAt,desc (Spring Data format)
Correlation ID	X-Correlation-ID header echoed in all responses for tracing

2. Authentication and Security

2.1 Authentication mechanism

The LMS API uses JWT (JSON Web Token) Bearer token authentication. Tokens are issued by the Auth service on successful login. All protected endpoints require the Authorization header. Public endpoints (register, login, published course catalogue) do not require a token.

Token structure

Token type	Expiry	Storage	Notes
Access token	15 minutes	Memory (not localStorage)	Sent in Authorization header on every request
Refresh token	7 days	HttpOnly cookie	Used to obtain a new access token silently

Authorization header format

`Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`

JWT payload claims

Claim	Type	Description
sub	string	UserId — the authenticated user's ID
role	string	User role: ADMIN INSTRUCTOR STUDENT
email	string	User's email address
iat	number	Issued at (Unix timestamp)
exp	number	Expiry (Unix timestamp)
jti	string	Unique token ID — used for refresh token rotation and revocation

2.2 Role-based access control

Role	Scope	Restrictions
ADMIN	Full access to all endpoints	Can manage all users, approve courses, view all logs and reports
INSTRUCTOR	Own courses and students	Can only manage courses where InstructorId = their UserId; can only grade submissions for their courses
STUDENT	Own data and published courses	Can only view own submissions, progress, and grades; can only enrol in Published courses; cannot access admin or instructor management endpoints

Role	Scope	Restrictions
Public (no token)	Read-only published content	Can only access GET /api/courses and GET /api/courses/{id} (Published only)

2.3 Security headers

Header	Value	Purpose
Strict-Transport-Security	max-age=31536000; includeSubDomains	Enforce HTTPS
X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-Frame-Options	DENY	Prevent clickjacking
Content-Security-Policy	default-src 'self'	Restrict resource loading
X-Correlation-ID	<uuid>	Request tracing — echoed from request or generated
Retry-After	<seconds>	Included on 429 responses — tells client when to retry

2.4 Rate limiting

Limit type	Threshold	Response
Per IP (unauthenticated)	100 requests / minute	429 Too Many Requests + Retry-After header
Per user (authenticated)	300 requests / minute	429 Too Many Requests + Retry-After header
Login attempts	5 failed attempts / 15 min	429 + account temporarily locked
File upload (submissions)	10 uploads / hour per student	429 Too Many Requests

3. API Endpoint Reference

3.1 Authentication endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register a new user account
POST	/api/auth/login	Public	Login and receive JWT tokens
POST	/api/auth/logout	JWT	Logout and invalidate refresh token

POST /api/auth/register — request body

RegisterRequest

Field	Type	Required	Description
username	string	Yes	3–50 characters, alphanumeric and underscore only
password	string	Yes	Minimum 8 characters, 1 uppercase, 1 number, 1 special char
email	string	Yes	Valid email format, must be unique
role	string	Yes	ADMIN INSTRUCTOR STUDENT

POST /api/auth/register — response (201 Created)

```
{
  "userId": 42,
  "username": "john_doe",
  "email": "john@example.com",
  "role": "STUDENT"
}
```

POST /api/auth/login — request body

LoginRequest

Field	Type	Required	Description
email	string	Yes	Registered email address
password	string	Yes	User's password (plaintext over HTTPS)

POST /api/auth/login — response (200 OK)

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "userId": 42,
}
```



```
  "role": "STUDENT"
}
```

3.2 Course management endpoints

Method	Endpoint	Auth	Description
GET	/api/courses	Public	List published courses (paginated, filterable)
GET	/api/courses/{id}	Public	Get course details by ID
POST	/api/courses	Instructor	Create a new course (status=DRAFT)
PUT	/api/courses/{id}	Instructor	Update course details
DELETE	/api/courses/{id}	Instructor/Admin	Archive course (status=ARCHIVED)
PUT	/api/courses/{id}/publish	Instructor/Admin	Submit for approval or approve

GET /api/courses — query parameters

Parameter	Type	Default	Description
page	integer	0	Zero-indexed page number
size	integer	20	Results per page (max 100)
sort	string	createdAt,desc	Sort field and direction
category	string	—	Filter by category name
difficulty	string	—	BEGINNER INTERMEDIATE ADVANCED
instructorId	integer	—	Filter by instructor user ID
search	string	—	Full-text search on title and description
minDuration	integer	—	Minimum duration in hours
maxDuration	integer	—	Maximum duration in hours
maxPrice	decimal	—	Maximum price (0 = free only)

POST /api/courses — request body

CreateCourseRequest

Field	Type	Required	Description
title	string	Yes	2–200 characters
description	string	No	Rich text or plain text course description
category	string	Yes	Predefined category name
difficulty	string	Yes	BEGINNER INTERMEDIATE ADVANCED

Field	Type	Required	Description
price	decimal	Yes	0.00 for free courses
duration	integer	Yes	Estimated duration in hours

Course response object

```
{
  "courseId": 10,
  "title": "Spring Boot Fundamentals",
  "description": "Learn Spring Boot from scratch...",
  "category": "Backend Development",
  "difficulty": "BEGINNER",
  "instructor": { "userId": 5, "username": "jane_smith" },
  "status": "PUBLISHED",
  "price": 0.00,
  "duration": 12,
  "moduleCount": 5,
  "enrollmentCount": 234,
  "createdAt": "2025-03-01T09:00:00Z"
}
```

3.3 Enrolment endpoints

Method	Endpoint	Auth	Description
POST	/api/enrollments	Student	Enrol in a course
GET	/api/enrollments?student_id={id}	JWT	Get all enrolments for a student
GET	/api/enrollments?course_id={id}	Instructor/Admin	Get all students enrolled in a course
DELETE	/api/enrollments/{id}	Student/Admin	Unenrol from a course

POST /api/enrollments — request body

EnrolmentRequest

Field	Type	Required	Description
courseId	integer	Yes	ID of the published course to enrol in
paymentToken	string	No	Payment gateway token (required for paid courses only)

POST /api/enrollments — response (201 Created)

```
{
  "enrollmentId": 88,
  "studentId": 42,
  "courseId": 10,
  "enrolledAt": "2025-06-15T10:30:00Z",
  "completionPercentage": 0.00,
  "completedAt": null
}
```

3.4 Assessment and submission endpoints

Method	Endpoint	Auth	Description
GET	/api/assessments?course_id={id}	JWT	List all assessments for a course
POST	/api/assessments	Instructor	Create a new assessment
PUT	/api/assessments/{id}	Instructor	Update assessment configuration
DELETE	/api/assessments/{id}	Instructor	Delete assessment
POST	/api/submissions	Student	Submit a quiz or assignment
GET	/api/submissions?assessment_id={id}	Instructor/Admin	Get all submissions for an assessment
GET	/api/submissions/{id}	JWT	Get a specific submission
PUT	/api/submissions/{id}/grade	Instructor	Grade submission and add feedback

POST /api/assessments — request body

CreateAssessmentRequest

Field	Type	Required	Description
courseId	integer	Yes	ID of the course this assessment belongs to
title	string	Yes	Assessment title
type	string	Yes	QUIZ ASSIGNMENT
maxScore	decimal	Yes	Maximum achievable score
timeLimitMinutes	integer	No	Null = untimed; required for timed quizzes
maxAttempts	integer	Yes	Maximum allowed attempts (minimum 1)

POST /api/submissions — request body

SubmissionRequest

Field	Type	Required	Description
assessmentId	integer	Yes	ID of the assessment being submitted
answers	object[]	No	Array of {questionId, answer} for quiz submissions
fileUrl	string	No	Pre-uploaded S3 URL for assignment file submissions

POST /api/submissions — response (202 Accepted)

```
{
  "submissionId": 301,
  "assessmentId": 15,
  "studentId": 42,
  "submittedAt": "2025-06-15T11:00:00Z",
  "score": null,
  "status": "SUBMITTED"
}
```

Note: Score is null on submission. Auto-graded submissions will have score populated within seconds. Manual grading populates score when the instructor calls PUT /api/submissions/{id}/grade.

PUT /api/submissions/{id}/grade — request body

GradeRequest

Field	Type	Required	Description
score	decimal	Yes	Score awarded (must not exceed MaxScore)

Field	Type	Required	Description
feedback	string	No	Written feedback for the student (supports plain text)

3.5 Progress endpoints

Method	Endpoint	Auth	Description
GET	/api/progress?student_id={id}&course_id={id}	JWT	Get detailed progress for a student in a course
PUT	/api/progress/lesson-complete	Student	Mark a lesson as viewed / complete

GET /api/progress — response

```
{
  "studentId": 42,
  "courseId": 10,
  "completionPercentage": 65.00,
  "completedAt": null,
  "certificateUrl": null,
  "modules": [
    {
      "moduleId": 1,
      "title": "Getting Started",
      "completionPercentage": 100.00,
      "lessons": [
        { "lessonId": 1, "title": "Introduction", "viewed": true },
        { "lessonId": 2, "title": "Setup", "viewed": true }
      ]
    }
  ],
  "submissions": [
    { "assessmentId": 5, "score": 88.0, "maxScore": 100.0 }
  ]
}
```

PUT /api/progress/lesson-complete — request body

LessonCompleteRequest

Field	Type	Required	Description
enrollmentId	integer	Yes	ID of the student's enrolment record
lessonId	integer	Yes	ID of the lesson being marked complete

3.6 Log endpoints

Method	Endpoint	Auth	Description
GET	/api/logs	Admin	Get filtered list of logs (paginated)
GET	/api/logs/{id}	Admin	Get a specific log entry by ID
POST	/api/logs	System	Internal: write a new log entry

GET /api/logs — query parameters

Parameter	Type	Description
eventType	string	Filter by event type: LOGIN ENROLMENT LESSON_VIEW QUIZ_ATTEMPT SUBMISSION ERROR
actorId	integer	Filter by user (actor) ID
from	datetime	Start of date range (ISO 8601)
to	datetime	End of date range (ISO 8601)
page	integer	Zero-indexed page number (default 0)
size	integer	Results per page (default 20, max 100)

4. Error Handling

4.1 Error response format (RFC 7807)

All API errors return a JSON body conforming to RFC 7807 Problem Detail. This provides a consistent, machine-readable error format across all endpoints.

```
{
  "type": "https://api.lms.example.com/errors/resource-not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Course with ID 99 does not exist or is not published",
  "instance": "/api/courses/99",
  "correlationId": "a3f2bc9d-11ef-4a2b-9c3d-000000000001",
  "timestamp": "2025-06-15T11:30:00Z"
}
```

4.2 HTTP status codes

Status code	Meaning	When used
200 OK	Success	GET, PUT requests completed successfully
201 Created	Resource created	POST requests that create a new resource; Location header included
202 Accepted	Accepted for async processing	POST /api/submissions — submission saved, grading is async
204 No Content	Success, no body	DELETE requests completed successfully
400 Bad Request	Validation error	Request body fails Bean Validation; missing required fields
401 Unauthorized	Authentication required	Missing or invalid JWT token
402 Payment Required	Payment needed	Attempting to enrol in a paid course without payment
403 Forbidden	Insufficient permissions	Valid token but RBAC denies the operation
404 Not Found	Resource not found	Entity does not exist or is not visible to the caller
409 Conflict	Duplicate / state conflict	Enrolling in already-enrolled course; duplicate username/email
422 Unprocessable Entity	Business rule violation	Exceeding MaxAttempts; submitting to a closed assessment
429 Too Many Requests	Rate limit exceeded	WAF or API Gateway throttle triggered; Retry-After header included

Status code	Meaning	When used
500 Internal Server Error	Unexpected error	Unhandled exception; logged with full stack trace and correlationId
502 Bad Gateway	External service failure	SES, S3, or payment gateway call failed

4.3 Validation error response

When a 400 is returned due to Bean Validation failures, the errors array contains field-level details:

```
{
  "type": "https://api.lms.example.com/errors/validation-failed",
  "title": "Validation failed",
  "status": 400,
  "detail": "Request body contains invalid fields",
  "errors": [
    { "field": "password", "message": "must be at least 8 characters" },
    { "field": "email", "message": "must be a valid email address" }
  ]
}
```

5. File Upload (Assignment Submissions)

5.1 Upload approach

File uploads for assignment submissions use a two-step S3 pre-signed URL approach. The client does not send files directly to the API; instead it obtains a pre-signed upload URL, uploads directly to S3 from the browser, then includes the resulting S3 URL in the submission request. This keeps large files off the application tier and satisfies the 30s content load SLA.

5.2 Upload flow

Step	Action	Endpoint / Service
1	Client requests pre-signed upload URL for a file	POST /api/files/upload-url (not in scope v1 — use AWS S3 SDK directly from frontend with Cognito Identity Pool temp credentials)
2	Client uploads file directly to S3	PUT <presigned-url> (direct to S3, bypasses API)
3	Client submits assessment with S3 file URL	POST /api/submissions with fileUrl field populated
4	Assessment service validates S3 URL format	Checks URL matches expected S3 bucket and path pattern
5	Submission persisted with fileUrl	Instructor accesses file via pre-signed download URL generated on demand

5.3 Accepted file types and size limits

File type	MIME type	Max size
PDF	application/pdf	20 MB
Word document	application/vnd.openxmlformats-officedocument.wordprocessingml.document	20 MB
Image (JPEG)	image/jpeg	10 MB
Image (PNG)	image/png	10 MB

6. WebSocket API (Real-time Notifications)

6.1 Overview

Real-time in-app notifications are delivered over a WebSocket connection managed by Amazon API Gateway WebSocket API. The client establishes a persistent WebSocket connection after login. The Notification service pushes events to connected clients without polling.

6.2 Connection

```
wss://ws.lms.example.com/ws?token=<accessToken>
```

The access token is passed as a query parameter on the initial WebSocket handshake. The API Gateway Lambda authoriser validates the token before allowing the connection. Connections are automatically closed after 2 hours of inactivity (API Gateway default).

6.3 Notification message format

```
{
  "type": "GRADE_RELEASED",
  "payload": {
    "submissionId": 301,
    "assessmentTitle": "Week 3 Quiz",
    "score": 88.0,
    "maxScore": 100.0,
    "feedback": "Good work on the theory section!"
  },
  "timestamp": "2025-06-15T11:45:00Z"
}
```

6.4 Notification types

Type	Trigger	Payload fields
GRADE_RELEASED	Instructor grades a submission	submissionId, assessmentTitle, score, maxScore, feedback
ENROLMENT_CONFIRMED	Student successfully enrolls	courseId, courseTitle, enrolledAt
NEW_CONTENT	Instructor publishes new lesson	courseId, moduleTitle, lessonTitle, lessonId
DEADLINE_REMINDER	48 hours before assessment due	assessmentId, assessmentTitle, dueAt
ANNOUNCEMENT	Instructor posts course announcement	courseId, courseTitle, announcementText, postedAt

Type	Trigger	Payload fields
CERTIFICATE_READY	Course completion reaches 100%	courseId, courseTitle, certificateUrl

7. Pagination and Filtering

7.1 Pagination

All list endpoints support page-based pagination using Spring Data's Pageable convention. The response envelope wraps the data array with pagination metadata.

Standard list response envelope

```
{
  "content": [ ...items... ],
  "page": {
    "size": 20,
    "number": 0,
    "totalElements": 157,
    "totalPages": 8
  }
}
```

Pagination query parameters

Parameter	Default	Max	Description
page	0	—	Zero-indexed page number
size	20	100	Number of results per page
sort	createdAt,desc	—	Field name and direction (asc or desc)

7.2 Filtering conventions

Pattern	Example	Notes
Exact match	?role=INSTRUCTOR	Enum values; case-insensitive
Range filter	?from=2025-01-01T00:00:00Z&to=2025-06-30T23:59:59Z	ISO 8601 datetimes
Numeric range	?minDuration=5&maxDuration=20	Integer values
Full-text search	?search=spring+boot	URL-encoded; searches title and description
Multiple values	?category=Backend&category=Frontend	Repeated parameter = OR filter

8. Complete API Quick Reference

Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login, receive JWT
POST	/api/auth/logout	JWT	Logout, revoke token

Users

Method	Endpoint	Auth	Description
GET	/api/users	Admin	List all users
GET	/api/users/{id}	JWT	Get user profile
PUT	/api/users/{id}	JWT	Update user profile
DELETE	/api/users/{id}	Admin	Deactivate user

Courses

Method	Endpoint	Auth	Description
GET	/api/courses	Public	Published catalogue
GET	/api/courses/{id}	Public	Course detail
POST	/api/courses	Instructor	Create course (DRAFT)
PUT	/api/courses/{id}	Instructor	Update course
DELETE	/api/courses/{id}	Instructor/Admin	Archive course
PUT	/api/courses/{id}/publish	Instructor/Admin	Submit or approve

Modules and lessons

Method	Endpoint	Auth	Description
GET	/api/courses/{id}/modules	JWT	List course modules
POST	/api/modules	Instructor	Create module
PUT	/api/modules/{id}	Instructor	Update module
DELETE	/api/modules/{id}	Instructor	Delete module (cascades)
GET	/api/modules/{id}/lessons	JWT	List module lessons
POST	/api/lessons	Instructor	Create lesson

Method	Endpoint	Auth	Description
PUT	/api/lessons/{id}	Instructor	Update lesson
DELETE	/api/lessons/{id}	Instructor	Delete lesson

Enrolment

Method	Endpoint	Auth	Description
POST	/api/enrollments	Student	Enrol in course
GET	/api/enrollments?student_id={id}	JWT	Student's courses
GET	/api/enrollments?course_id={id}	Instructor/Admin	Course roster
DELETE	/api/enrollments/{id}	Student/Admin	Unenrol

Assessments

Method	Endpoint	Auth	Description
GET	/api/assessments?course_id={id}	JWT	Course assessments
POST	/api/assessments	Instructor	Create assessment
PUT	/api/assessments/{id}	Instructor	Update assessment
DELETE	/api/assessments/{id}	Instructor	Delete assessment

Submissions

Method	Endpoint	Auth	Description
POST	/api/submissions	Student	Submit quiz or assignment
GET	/api/submissions?assessment_id={id}	Instructor/Admin	All submissions
GET	/api/submissions/{id}	JWT	Single submission
PUT	/api/submissions/{id}/grade	Instructor	Grade + feedback

Progress

Method	Endpoint	Auth	Description
GET	/api/progress?student_id={id}&course_id={id}	JWT	Progress detail
PUT	/api/progress/lesson-complete	Student	Mark lesson complete

Logs

Method	Endpoint	Auth	Description
GET	/api/logs	Admin	Filtered log list
GET	/api/logs/{id}	Admin	Single log entry
POST	/api/logs	System	Internal log write

9. API Changelog

Version	Date	Changes
1.0	June 2026	Initial release — all endpoints for Auth, User, Course, Module, Lesson, Enrolment, Assessment, Submission, Progress, Logging